

RELATÓRIO DO PROJETO

Projeto POO – Amv Eventos

Programação Orientada a Objetos em Python

Sistema para organizar a logística de entrega de materiais de cozinha em eventos: motoristas, veículos, materiais, itens de entrega e o andamento de cada evento.

INTEGRANTES

João
David
Renan

Setembro de 2026

Sumário

1. Justificativa	3
2. Funcionalidades	3
3. Temas abordados	4
4. Bibliotecas	6
5. Tecnologias	7
6. Distribuição de tarefas	7
7. Repositório	7

1. Justificativa

Eventos que dependem de estrutura de cozinha, como ações realizadas em cozinhas comunitárias, precisam que materiais específicos (panelas industriais, utensílios e equipamentos) cheguem ao local certo, na quantidade certa e no dia combinado. Quando essa logística é controlada de forma manual, em anotações ou planilhas soltas, é comum perder informações importantes: qual motorista ficou responsável, qual veículo foi usado, quais materiais foram enviados e em que quantidade.

O projeto **Amv Eventos** foi desenvolvido para modelar essa logística em Python, reunindo em um único sistema os dados de motoristas, veículos, materiais de cozinha, itens de entrega e eventos. Cada evento registra a data, o destino, o motorista e o veículo responsáveis e a lista de itens transportados, além de um status que acompanha o seu andamento do início ao fim.

Do ponto de vista acadêmico, o cenário foi escolhido por ser próximo de uma situação real e por exigir, de forma natural, os principais conceitos de Programação Orientada a Objetos. Cada entidade do problema se tornou uma classe, e as relações entre elas (um motorista atende vários eventos; um evento reúne vários itens) foram primeiro representadas no diagrama de classes e depois implementadas no código. Assim, o projeto permitiu praticar o caminho completo entre a modelagem e a implementação.

2. Funcionalidades

O sistema é composto por cinco classes. Cada uma concentra os dados de uma entidade e as operações que fazem sentido sobre ela, conforme resumido abaixo.

2.1 Funcionalidades por classe

Classe	O que faz
Motorista	Cadastra e atualiza os dados do motorista (nome, CPF e CNH). Na atualização, apenas os campos informados são alterados.
Veiculo	Cadastra e atualiza placa, modelo e capacidade do veículo. Recusa capacidade igual a zero ou negativa.
MaterialCozinha	Cadastra e atualiza os materiais transportados (nome, tipo e descrição), como uma panela industrial de inox.
ItemEntrega	Associa um material a uma quantidade e permite adicionar ou remover unidades. Recusa valores não positivos e a remoção de mais itens do que os disponíveis.
Evento	Reúne data, destino, motorista, veículo e a lista de itens da entrega. Permite incluir itens com <code>adicionar_item()</code> e controla o andamento do evento pelo status.

2.2 Ciclo de vida do evento

Todo evento começa com o status **Pendente**. O método `criar()` o coloca como **Criado**; a partir daí, `finalizar()` encerra o evento como **Finalizado** e `cancelar()` o registra como **Cancelado**. Cada mudança é confirmada com uma mensagem no terminal.

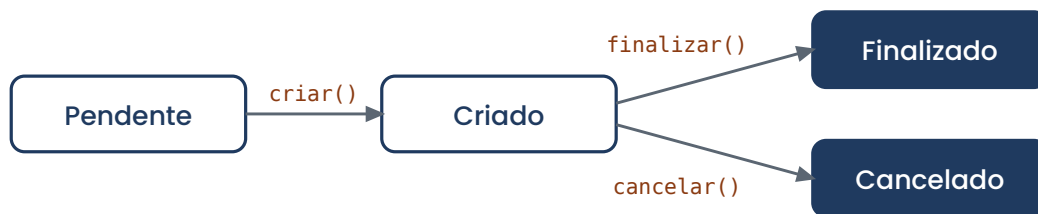


Figura 1 – Fluxo de status previsto para um evento.

2.3 Demonstração de execução

O arquivo do sistema inclui um teste de funcionalidade, executado quando o programa é rodado diretamente. Ele cadastra o motorista Carlos Silva e um veículo Kombi, cria o material Panela Industrial e um item de entrega com 5 unidades, às quais são adicionadas mais 10. Em seguida, monta um evento com destino à Cozinha Comunitária Centro, vincula o item, cria e finaliza o evento. A saída obtida foi:

```
Adicionados 10 itens. Total atual: 15
Evento 1 para Cozinha Comunitária Centro foi criado.
Status atual do evento: Criado
Evento 1 finalizado.
Status atual do evento: Finalizado
```

3. Temas abordados

O projeto aplica os conceitos centrais de Programação Orientada a Objetos e de modelagem UML. A tabela a seguir relaciona cada tema ao ponto do projeto em que ele aparece.

Tema	Onde aparece no projeto
Classes e objetos	As cinco entidades do problema viraram classes; o teste de funcionalidade cria objetos concretos a partir delas (<code>m1</code> , <code>v1</code> , <code>mat1</code> , <code>item1</code> , <code>ev1</code>).
Construtor e atributos	O método <code>__init__</code> recebe os dados iniciais e os guarda em atributos por meio de <code>self</code> (por exemplo, <code>self.placa</code> e <code>self.cnh</code>).
Métodos	Cada classe expõe seus comportamentos: cadastrar e atualizar nos cadastros, adicionar e remover no item de entrega, criar, finalizar e cancelar no evento.

Tema	Onde aparece no projeto
Associação	Evento guarda referências para um Motorista e um Veiculo; ItemEntrega guarda uma referência para o MaterialCozinha correspondente.
Agregação	Evento mantém uma lista de itens de entrega (<code>self.itens</code>), preenchida pelo método <code>adicionar_item()</code> .
Encapsulamento de regras	As validações ficam dentro da classe dona do dado, e não espalhadas pelo programa: Veiculo protege a capacidade e ItemEntrega protege a quantidade.
Tratamento de exceções	Entradas inválidas geram <code>ValueError</code> com mensagem explicativa, impedindo que o objeto fique em um estado inconsistente.
Parâmetros opcionais	Os métodos <code>atualizar()</code> usam valor padrão <code>None</code> , o que permite alterar só os campos informados.
Controle de estado	O atributo <code>status</code> do Evento registra em que etapa ele se encontra.
Modelagem UML	Diagrama de classes com atributos, tipos, visibilidade pública (+), métodos, associações, agregação e multiplicidades.
Ponto de entrada	O bloco <code>if __name__ == "__main__":</code> executa o teste apenas quando o arquivo é rodado diretamente, e não quando é importado.

3.1 Diagrama de classes

A modelagem foi feita antes da implementação e serviu de guia para o código. O diagrama representa as cinco classes do sistema e as relações entre elas, resumidas na tabela abaixo; o diagrama completo é apresentado na Figura 2.

Relação	Multiplicidade	Significado
Motorista e Evento	1 para 0..*	Um motorista pode conduzir vários eventos; cada evento tem um motorista responsável.
Veiculo e Evento	1 para 0..*	Um veículo pode ser usado em vários eventos; cada evento utiliza um veículo.
Evento e ItemEntrega (agregação)	1 para 0..*	Um evento reúne vários itens de entrega.
ItemEntrega e MaterialCozinha	0..* para 1	Cada item se refere a um material; um mesmo material pode aparecer em vários itens.

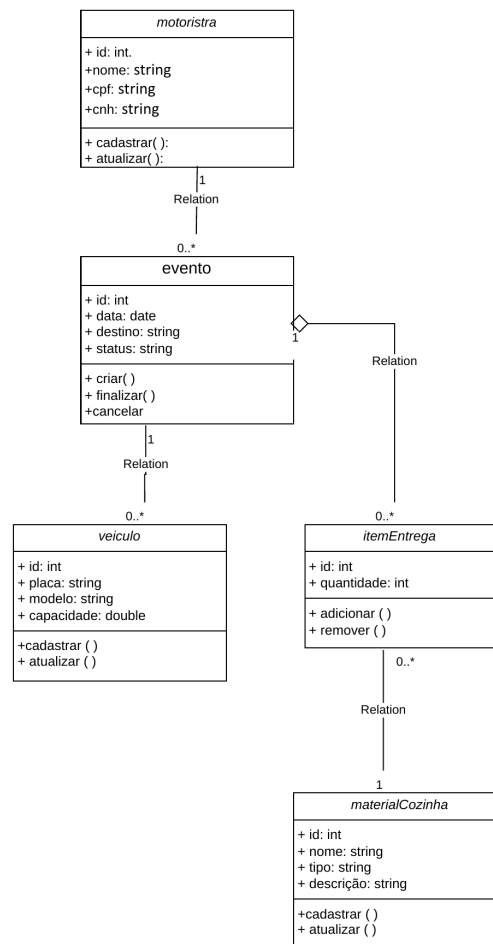


Figura 2 – Diagrama de classes do sistema, elaborado no draw.io.

4. Bibliotecas

O projeto **não utiliza bibliotecas externas** nem módulos importados: todo o código foi escrito apenas com recursos nativos da linguagem Python. Por isso, não há dependências a instalar nem arquivo *requirements.txt*; basta ter o Python 3 instalado para executar o sistema. Os recursos nativos utilizados foram:

Recurso nativo	Uso no projeto
<code>class</code> e <code>__init__</code>	Definição das classes e inicialização dos objetos.
<code>list</code>	Armazenamento dos itens de cada evento (<code>self.itens</code>).
<code>ValueError</code>	Sinalização de dados inválidos, como capacidade ou quantidade não positivas.
f-strings	Montagem das mensagens exibidas ao usuário.
<code>print()</code>	Exibição das mensagens de confirmação no terminal.

5. Tecnologias

As tecnologias e ferramentas empregadas no desenvolvimento foram:

Tecnologia	Uso no projeto
Python 3	Linguagem de programação usada na implementação do sistema.
UML	Notação utilizada para modelar as classes e seus relacionamentos.
draw.io (diagrams.net)	Ferramenta usada para desenhar o diagrama de classes.
Git	Controle de versão do código e da documentação.
GitHub	Hospedagem do repositório e colaboração entre os integrantes.
Markdown	Escrita da documentação do repositório (README.md).

6. Distribuição de tarefas

Cada integrante ficou com uma frente principal de trabalho, mas o código e o diagrama passaram pela revisão de toda a equipe, que corrigiu os pontos necessários antes da versão final. O repositório foi construído em conjunto.

Integrante	Responsabilidade principal	Observação
João	Desenvolvimento do código em Python.	Código revisado e corrigido por toda a equipe.
David	Elaboração do diagrama de classes (UML).	Diagrama revisado e corrigido por toda a equipe.
Renan	Elaboração do relatório e da documentação do repositório.	Inclui o arquivo README.md.
Equipe	Criação e organização do repositório no GitHub.	Trabalho feito em conjunto por todos os integrantes.

7. Repositório

O código-fonte, o diagrama de classes, este relatório e a documentação do projeto estão disponíveis no repositório do GitHub:

<https://github.com/renan2026/Projeto-POO-AMV-Eventos>